

DBMS — Module 4

Database Design Concepts

VVI Exam Questions with Complete Answers

Total Questions	15	Topics	Schema Refinement, FD, Normalization, BCNF, MVD, 4NF, 5NF
Max Marks	~80	Difficulty	Easy to Hard (all covered)

Q1

What is Schema Refinement? Explain the different types of anomalies in a database with examples.

[5 Marks]

Answer:

Schema Refinement is the process of analysing and improving a database schema to eliminate data redundancy, anomalies, and update inconsistencies. The goal is to achieve a clean, well-structured schema that stores each piece of information exactly once.

Database Anomalies

When a relation is not properly designed, three types of anomalies can occur:

Anomaly	What Happens	Example (EmpDept table)
Insertion Anomaly	Cannot insert data without providing unnecessary information	Cannot add a new department unless at least one employee is assigned to it.
Deletion Anomaly	Deleting a row unintentionally destroys other important data.	Deleting the last employee in a department also deletes the department info.
Update Anomaly	Changing one fact requires updating multiple rows, leading to inconsistency.	Changing a department name requires updating every employee row in that department.

Solution: Decompose the relation into smaller, well-structured tables using Normalization to remove these anomalies.

Q2

Define Functional Dependency. Explain Trivial and Non-Trivial FDs with examples.

[4 Marks]

Answer:

Functional Dependency (FD): In a relation R, an attribute Y is functionally dependent on attribute X (written $X \rightarrow Y$) if and only if for every valid relation instance, whenever two tuples agree on X, they must also agree on Y. In simple terms: knowing X uniquely determines Y.

Example: In Employee(EmpID, Name, DeptID, DeptName) — $\text{EmpID} \rightarrow \text{Name}$ means knowing EmpID uniquely gives the employee's name.

Type	Definition	Example
Trivial FD	Y is a subset of X (always true by definition)	$\{A, B\} \rightarrow A$ or $\{A, B\} \rightarrow B$
Non-Trivial FD	Y is NOT a subset of X (carries real information)	$\text{EmpID} \rightarrow \text{EmpName}$ (Name is not part of EmpID)
Partial FD	Non-key attribute depends on part of a composite key	$(\text{SID}, \text{CID}) \rightarrow \text{Grade}$, $\text{SID} \rightarrow \text{SName}$ is partial
Transitive FD	$A \rightarrow B$ and $B \rightarrow C$ implies $A \rightarrow C$ where B is non-key	$\text{EmpID} \rightarrow \text{DeptID}$ and $\text{DeptID} \rightarrow \text{DeptName} \Rightarrow \text{EmpID} \rightarrow \text{DeptName}$
Full FD	Non-key attr depends on the WHOLE primary key	$(\text{SID}, \text{CID}) \rightarrow \text{Grade}$ — Grade depends on both

Q3

State and explain Armstrong's Axioms with examples. Also state derived rules.

[6 Marks]

Answer:

Armstrong's Axioms are a set of sound and complete inference rules used to derive all functional dependencies from a given set F.

Primary (Sound & Complete) Axioms:

Axiom	Rule	Example
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$	$\{A, B\} \rightarrow A$
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$ (add same attrs to both sides)	$A \rightarrow B$ implies $AC \rightarrow BC$
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	$A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C$

Derived Rules:

Rule	Derivation
Union: $X \rightarrow Y$ and $X \rightarrow Z \Rightarrow X \rightarrow YZ$	Augment $X \rightarrow Y$ to get $XZ \rightarrow YZ$; augment $X \rightarrow Z$ to get $XZ \rightarrow YZ$; use transitivity
Decomposition: $X \rightarrow YZ \Rightarrow X \rightarrow Y$ and $X \rightarrow Z$	Reflexivity gives $YZ \rightarrow Y$; then transitivity with $X \rightarrow YZ$
Pseudotransitivity: $X \rightarrow Y$ and $WY \rightarrow Z \Rightarrow WX \rightarrow Z$	Augment $X \rightarrow Y$ to $WX \rightarrow WY$; then transitivity with $WY \rightarrow Z$

Q4

Explain the process of Normalization. What are 1NF, 2NF, 3NF, and BCNF? Distinguish between 3NF and BCNF.

[10 Marks ★★★]

Answer:

Normalization is a systematic process of organizing a relational database schema to reduce data redundancy and improve data integrity by decomposing large tables into smaller, well-structured tables using Functional Dependencies.

Normal Form	Required Condition	Anomaly Removed
1NF	All attributes must be atomic (single-valued, no repeating groups)	Multi-valued / composite attribute issues
2NF	Must be in 1NF + No partial dependency (every non-prime attribute fully depends on the whole PK)	Partial dependencies
3NF	Must be in 2NF + No transitive dependency (non-prime attribute does not depend on another non-prime attribute)	Transitive dependencies
BCNF	Must be in 3NF + For every non-trivial FD $X \rightarrow Y$, X must be a superkey	All remaining anomalies that 3NF misses

Step-by-step Normalization Example:

Starting Relation:

`ORDER(OrdID, OrdDate, CustID, CustName, ProdID, ProdName, Qty, UnitPrice)`

FDs: $\text{OrdID} \rightarrow \text{OrdDate}$, $\text{CustID} \rightarrow \text{CustName}$; $\text{CustID} \rightarrow \text{ProdName}$, $\text{ProdID} \rightarrow \text{ProdName}$, UnitPrice ; $(\text{OrdID}, \text{ProdID}) \rightarrow \text{Qty}$. Primary Key = (OrdID, ProdID)

- **1NF:** All values are already atomic. Passes 1NF.
- **2NF:** CustName depends only on CustID (partial dep), ProdName/UnitPrice on ProdID (partial dep). Decompose:

`ORDER_MAIN(OrdID, OrdDate, CustID) | CUSTOMER(CustID, CustName)`

`PRODUCT(ProdID, ProdName, UnitPrice) | ORDER_DETAIL(OrdID, ProdID, Qty)`

- **3NF:** CustName depended on CustID which depended on OrdID — transitive dep already removed above. Now in 3NF.
- **BCNF:** In each decomposed relation, the determinant is a superkey — so all relations are in BCNF.

3NF vs BCNF Comparison:

Point	3NF	BCNF
Condition	For $X \rightarrow Y$: X is superkey OR Y is a prime attribute	For $X \rightarrow Y$: X MUST be a superkey (no exception)
Strictness	Less strict	Stricter than 3NF
Lossless?	Yes	Yes
Dep. Preserving?	Always Yes	Not always guaranteed
When to use?	When dep. preservation is required	When eliminating all redundancy is priority
Counter-example	$R(A,B,C): A \rightarrow B, B \rightarrow C, C \rightarrow A$ — in 3NF	Same R may not satisfy BCNF if B is not a superkey

Q5

Consider $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $CD \rightarrow E$, $B \rightarrow D$, $E \rightarrow A$. Find candidate keys and normalize to BCNF.

[8 Marks ★★★]

Answer:

Step 1 — Find Attribute Closures:

Attribute	Closure	Is it a Superkey?
-----------	---------	-------------------

A+	A→BC, B→D, so A+ = {A,B,C,D}. CD→E: CD already derived. A+ = {A,B,C,D,E}	YES — Candidate Key
B+	B→D. B+ = {B,D}	No
C+	C+ = {C}	No
D+	D+ = {D}	No
E+	E→A, A→BC, B→D. E+ = {E,A,B,C,D} = {A,B,C,D,E}	YES — Candidate Key
CD+	CD→E, E→A, A→BC, B→D. CD+ = {A,B,C,D,E}	YES — Candidate Key

Candidate Keys: A, E, CD

Prime attributes: A, C, D, E | Non-prime: B

Step 2 — Check BCNF Violations:

- A→BC: A is a superkey ✓ (No violation)
- CD→E: CD is a superkey ✓ (No violation)
- B→D: B is NOT a superkey ✗ (BCNF VIOLATION!)
- E→A: E is a superkey ✓ (No violation)

Step 3 — Decompose to remove B→D violation:

R1(B, D) — covers FD B→D

R2(A, B, C, E) — remaining attributes

Check R2: FDs in R2 — A→BC (A is key in R2 ✓), E→A (E is key in R2 ✓). R2 is in BCNF.

Final BCNF Relations: R1(B,D) and R2(A,B,C,E)

Q6

What is Lossless Join Decomposition? How do you test it? Give an example.

[6 Marks]

Answer:

Lossless Join Decomposition: A decomposition of relation R into R1 and R2 is lossless if the natural join of R1 and R2 gives back exactly R — no extra (spurious) tuples are added and no original tuples are lost.

Condition (for binary decomposition):

$R1 \cap R2 \rightarrow R1$ OR $R1 \cap R2 \rightarrow R2$

i.e., the common attributes (intersection) must be a superkey of at least one of R1 or R2.

Example:

R(A, B, C) with FD: $A \rightarrow B$

Decomposition 1: R1(A,B), R2(A,C)

$R1 \cap R2 = \{A\}$. Since $A \rightarrow B$ (A is key of R1), decomposition is LOSSLESS ✓

Decomposition 2: R1(A,B), R2(B,C)

$R1 \cap R2 = \{B\}$. $B \rightarrow A$? No. $B \rightarrow C$? Not given. So this is LOSSY (spurious tuples possible) ✗

Q7

Explain Dependency Preserving Decomposition with an example where BCNF fails to preserve dependencies.

[6 Marks]

Answer:

Dependency Preserving Decomposition: A decomposition $\{R1, R2, \dots, Rn\}$ of R is dependency preserving if every original FD in F can be derived from the FDs within the individual Ri relations. Formally: $(F1 \cup F2 \cup \dots \cup Fn)^+ = F^+$

This is important because if a dependency is not preserved, we cannot enforce that constraint without joining relations, which is expensive.

Example where BCNF loses dependency:

R(Student, Subject, Teacher) with FDs:

$(\text{Student, Subject}) \rightarrow \text{Teacher}$ [A student takes a subject from one teacher]

$\text{Teacher} \rightarrow \text{Subject}$ [A teacher teaches only one subject]

Candidate Keys: (Student, Subject) and (Student, Teacher)

BCNF Violation: $\text{Teacher} \rightarrow \text{Subject}$, but Teacher is not a superkey.

BCNF Decomposition:

R1(Teacher, Subject) – covers $\text{Teacher} \rightarrow \text{Subject}$

R2(Student, Teacher) – remaining

Problem: The FD $(\text{Student, Subject}) \rightarrow \text{Teacher}$ CANNOT be verified in R1 or R2 alone. We'd need to join them. So BCNF decomposition is NOT dependency-preserving here.

3NF preserves this dependency — keeps R(Student, Subject, Teacher) intact in 3NF since Teacher is a prime attribute.

Q8

What are the differences between Lossless Join Decomposition and Dependency Preserving Decomposition?

[4 Marks]

Answer:

Point	Lossless Join Decomp.	Dependency Preserving Decomp.
Definition	Natural join of sub-relations reconstructs the original relation exactly	All original FDs can be checked/enforced within individual sub-relations
Concern	Prevents spurious (extra/wrong) tuples on joining back	Prevents loss of integrity constraints during decomposition
Test	$R1 \bowtie R2 \rightarrow R1 \text{ or } R2$	$(F1 \cup F2 \cup \dots)^+ \text{ must equal } F^+$
BCNF	Always gives lossless decomposition	May NOT preserve all FDs
3NF	Always gives lossless decomposition	Always dependency preserving
Which to prefer?	Both are desirable; lossless is non-negotiable	3NF preferred when both are needed

Q9

Define Multivalued Dependency (MVD). Explain 4NF with a suitable example.

[6 Marks]

Answer:

Multivalued Dependency (MVD): In a relation $R(X, Y, Z)$, a Multivalued Dependency $X \twoheadrightarrow Y$ exists if for every pair of tuples t_1 and t_2 in R where $t_1[X] = t_2[X]$, there exist tuples t_3 and t_4 such that:

$$t_3[X] = t_4[X] = t_1[X]$$

$$t_3[Y] = t_1[Y], t_3[Z] = t_2[Z]$$

$$t_4[Y] = t_2[Y], t_4[Z] = t_1[Z]$$

In simpler terms: the set of Y -values associated with X is independent of the Z -values.

Example:

Consider $EMP_SKILL_DEPT(EMPID, Skill, Dependent)$:

EmpID	Skill	Dependent
E1	Java	Ravi
E1	Java	Sita
E1	Python	Ravi
E1	Python	Sita

Here $EMPID \twoheadrightarrow Skill$ and $EMPID \twoheadrightarrow Dependent$ independently. This causes 4 rows even though there are only 2 skills and 2 dependents — redundancy exists even in BCNF!

4NF Definition:

A relation R is in 4NF if for every non-trivial MVD $X \twoheadrightarrow Y$ in R , X is a superkey of R .

Converting to 4NF — Decompose:

$EMP_SKILL(EMPID, Skill)$ — covers $EMPID \twoheadrightarrow Skill$

$EMP_DEPENDENT(EMPID, Dependent)$ — covers $EMPID \twoheadrightarrow Dependent$

Now each relation contains only one multi-valued fact about $EMPID$ — so both are in 4NF.

Q10

Explain Join Dependency and 5NF (Project-Join Normal Form) with a suitable example.

[5 Marks]

Answer:

Join Dependency (JD): A relation R satisfies a join dependency $*(R_1, R_2, \dots, R_n)$ if R can be reconstructed by the natural join of its projections $R_1, R_2, \dots, R_n$. MVDs are a special case of JDs (binary decomposition).

5NF (PJNF): A relation R is in 5NF if for every join dependency $*(R_1, \dots, R_n)$ implied by R , every R_i is a superkey of R . 5NF removes all redundancy that arises from join dependencies not implied by candidate keys.

Classic SPJ Example:

$SUPPLY(Supplier, Product, Project)$ — a supplier supplies a product used in a project.

Supplier	Product	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1

S2	P1	J1
----	----	----

If the business rule says: 'whenever S supplies P, and P is used in J, and S is associated with J, then S supplies P for J' — this is a join dependency $\Join(SP, PJ, SJ)$. Decomposing into 3 pairwise tables and joining gives back SUPPLY without spurious tuples. This means SUPPLY satisfies 5NF via that JD.

Q11

Normalize the following relation to 3NF: Emp(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, ProjectID, ProjectName) FDs: EmpID→EmpName,DeptID; DeptID→DeptName,ManagerID; ManagerID→ManagerName; ProjectID→ProjectName

[10 Marks ★★★]

Answer:

Step 1 — Identify Primary Key and check 1NF:

Primary Key = (EmpID, ProjectID). All attributes are atomic, so relation is in 1NF.

Step 2 — Check for Partial Dependencies (2NF):

FD	Depends on	Type
EmpID→EmpName, DeptID	Part of PK (EmpID only)	PARTIAL DEPENDENCY ✗
DeptID→DeptName, ManagerID	Non-key attribute	TRANSITIVE (via EmpID) ✗
ManagerID→ManagerName	Non-key attribute	TRANSITIVE ✗
ProjectID→ProjectName	Part of PK (ProjectID only)	PARTIAL DEPENDENCY ✗
(EmpID, ProjectID) → ?	Whole PK	Only Qty/role if present — OK

Step 3 — Decompose to 2NF (remove partial dependencies):

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID) [Key: EmpID]

PROJECT(ProjectID, ProjectName) [Key: ProjectID]

EMP_PROJECT(EmpID, ProjectID) [Key: EmpID+ProjectID]

Step 4 — Check for Transitive Dependencies (3NF):

In EMPLOYEE: EmpID→DeptID and DeptID→DeptName,ManagerID — transitive! Also EmpID→ManagerID and ManagerID→ManagerName — transitive!

Step 5 — Decompose to 3NF (remove transitive dependencies):

EMPLOYEE(EmpID, EmpName, DeptID) [Key: EmpID]

DEPARTMENT(DepID, DeptName, ManagerID) [Key: DeptID]

MANAGER(ManagerID, ManagerName) [Key: ManagerID]

PROJECT(ProjectID, ProjectName) [Key: ProjectID]

EMP_PROJECT(EmpID, ProjectID) [Key: EmpID+ProjectID]

Final 3NF Schema (5 tables):

Table	Attributes	Primary Key	FDs Covered
EMPLOYEE	EmpID, EmpName, DeptID	EmpID	EmpID→EmpName, DeptID
DEPARTMENT	DeptID, DeptName, ManagerID	DeptID	DeptID→DeptName, ManagerID
MANAGER	ManagerID, ManagerName	ManagerID	ManagerID→ManagerName
PROJECT	ProjectID, ProjectName	ProjectID	ProjectID→ProjectName
EMP_PROJECT	EmpID, ProjectID	(EmpID, ProjectID)	Assignment fact

Q12

What is BCNF? Why is every BCNF relation also in 3NF, but not vice versa? Give a counter-example.

[6 Marks]

Answer:

BCNF (Boyce-Codd Normal Form): A relation R is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$ in R, X must be a superkey of R. BCNF is a stronger version of 3NF — it removes the exception that 3NF allows for prime attributes.

Why every BCNF relation is in 3NF:

3NF condition: For $X \rightarrow Y$, either (a) X is a superkey, OR (b) Y is a prime attribute. BCNF requires (a) always. So if a relation satisfies BCNF (condition a always holds), it automatically satisfies 3NF as well (since condition a is a subset of 3NF conditions). Therefore: $BCNF \subset 3NF$ (every BCNF is 3NF, but not every 3NF is BCNF).

Counter-example (3NF but not BCNF):

R(Student, Course, Teacher) with FDs:

$(Student, Course) \rightarrow Teacher$ [A student in a course has one teacher]

$Teacher \rightarrow Course$ [Each teacher teaches only one course]

Candidate Keys: (Student, Course) and (Student, Teacher)

Prime attributes: Student, Course, Teacher (all are part of some candidate key)

3NF check on $Teacher \rightarrow Course$: Teacher is NOT a superkey, but Course IS a prime attribute — so 3NF condition (b) is satisfied. Relation is in 3NF.

BCNF check on $Teacher \rightarrow Course$: Teacher is NOT a superkey — BCNF VIOLATED. So this relation is in 3NF but NOT in BCNF.

Q13

Explain Closure of a set of Functional Dependencies. How is it used in finding candidate keys?

[5 Marks]

Answer:

Closure of F (F⁺): The closure of a set of FDs F is the set of ALL functional dependencies that can be logically derived from F using Armstrong's axioms. Computing full F⁺ is complex, so we use Attribute Closure instead.

Attribute Closure (X⁺): The attribute closure of a set of attributes X under F is the set of all attributes that X functionally determines (directly or transitively).

Algorithm to compute X⁺ :

```
result = X
repeat:
  for each FD Y→Z in F:
    if Y is a subset of result:
      result = result ∪ Z
until no change in result
X+ = result
```

Example: R(A,B,C,D), F = {A→B, B→C, C→D}

A⁺ : Start={A}. A→B: add B={A,B}. B→C: add C={A,B,C}. C→D: add D={A,B,C,D}. A⁺ = {A,B,C,D}

Since A⁺ = all attributes, A is a superkey (and candidate key, since no subset of {A} gives all attrs).

Using Attribute Closure to find Candidate Keys:

1. Compute X⁺ for each subset of attributes.
2. If X⁺ = all attributes of R, then X is a superkey.
3. X is a candidate key if it is a superkey AND no proper subset of X is also a superkey.

Q14

What is the difference between 3NF and BCNF? When would you prefer 3NF over BCNF?

[4 Marks]

Answer:

Aspect	3NF	BCNF
Condition	X→Y: X is superkey OR Y is prime attribute	X→Y: X MUST be a superkey (no exception)
Strictness	Less strict	Stricter
Redundancy	May have slight redundancy	Eliminates more redundancy
Lossless	Yes	Yes
Dep. Preserving	Always YES	NOT always
Anomalies	May retain some if Y is prime	Removes most update anomalies
Use case	When all FD constraints must be enforced locally	When maximum data integrity/no redundancy is needed

When to prefer 3NF over BCNF:

Prefer 3NF when BCNF decomposition causes loss of some functional dependencies (i.e., when dependency preservation is more important than complete elimination of redundancy). Example: the Teacher-Course-Student relation cannot be decomposed to BCNF without losing the FD (Student,

Course)→Teacher. In such cases, 3NF is the right choice.

Q15

Explain Minimal Cover (Canonical Cover) of a set of FDs with algorithm and example.

[6 Marks ★★]

Answer:

Canonical Cover (F_c): The canonical cover of a set of FDs F is a minimal/simplified equivalent set of FDs such that:

- No FD in F_c has extraneous (redundant) attributes on either side.
- No FD in F_c is redundant (can be derived from the rest of F_c).
- Every FD in F_c has a single attribute on the right-hand side.
- $F_c^+ = F^+$ (F_c is logically equivalent to F)

Algorithm to find Canonical Cover:

Step 1: Use decomposition rule – split all FDs to single RHS attribute

e.g., $A \rightarrow BC$ becomes $A \rightarrow B$ and $A \rightarrow C$

Step 2: Remove extraneous attributes from LHS

For $X \rightarrow Y$: if $X=AB$, check if A alone determines Y . If yes, remove B .

Step 3: Remove redundant FDs

For each FD $X \rightarrow Y$: check if $Y \in (F - \{X \rightarrow Y\})^+$. If yes, remove it.

Step 4: Merge FDs with same LHS back if needed

Example:

$F = \{ A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C \}$

Step 1 – Split: $A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C$

Step 2 – Remove duplicate $A \rightarrow B$ (appears twice): $A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C$

Step 3 – Check extraneous LHS in $AB \rightarrow C$:

Is $A \rightarrow C$ derivable without B ? A^+ under $\{A \rightarrow B, A \rightarrow C, B \rightarrow C\} = \{A, B, C\}$. Yes, $A \rightarrow C$ already. Remove $AB \rightarrow C$.

Step 4 – Check redundant FDs: $A \rightarrow C$: is C in A^+ under $\{A \rightarrow B, B \rightarrow C\}$? $A^+ = \{A, B, C\}$ YES. So $A \rightarrow C$ is redundant. Remove.

$F_c = \{ A \rightarrow B, B \rightarrow C \}$

Verification: F_c covers F^+ since $A \rightarrow BC$ is derived via $A \rightarrow B$ and $B \rightarrow C$ (transitivity). ✓

QUICK REVISION SHEET

Normal Form	Condition to Satisfy	Problem Removed
1NF	All attributes atomic, single-valued	Multi-value / composite attributes
2NF	1NF + No partial dependency on composite PK	Partial dependencies
3NF	2NF + No transitive dependency	Transitive dependencies
BCNF	For every $X \rightarrow Y$, X is a superkey	All anomalies 3NF misses
4NF	For every MVD $X \twoheadrightarrow Y$, X is a superkey	Multivalued dependency redundancy
5NF	For every JD $\ast(R_1..R_n)$, each R_i key is a superkey	Join dependency redundancy

Key Hierarchy: $1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF \supset 5NF$

Remember: BCNF is always lossless. 3NF is always lossless + dependency-preserving.

Attribute Closure X^+ = set of all attributes determined by X using given FDs.

Candidate Key = minimal set of attributes whose closure = all attributes.

All the Best for Your Exam! You've Got This! ■

DBMS — Module 4 | Database Design Concepts | Complete Q&A;